

SKALA 데이터분석 및 AIOps

[Day 1] 종합 실습 실행결과 보고서

실무형 수집·검증·품질 파이프라인

프로젝트명	Day 1 종합실습 - 비동기 API 수집·Pydantic 검증·CSV/Parquet 파이프라인
작성자	임유리
캠퍼스 / 반	판교 / 7반
작성일	2026-08-06

1. 프로젝트 개요

구현 목적	Open-Meteo(서울 3일 시간대별 기온·강수확률), Countries.dev(대한민국 국가 정보), ip-api(8.8.8.8 IP 위치 정보) 3개 API를 httpx + asyncio.gather()로 동시에 수집하고, Pydantic v2로 타입·범위를 검증한 뒤 CSV/Parquet 두 형식으로 저장하여 쓰기·읽기 성능을 비교한다.
사용 API	① Open-Meteo forecast API (서울 위경도 37.5665, 126.9780 / 3일 / 시간대별) ② Countries.dev alpha/KOR (대한민국 국가 정보) ③ ip-api json/8.8.8.8 (IP 기반 지역 정보)
기술 스택	Python, httpx(비동기 HTTP), asyncio, Pydantic v2, pandas + pyarrow(Parquet), pytest, pytest-asyncio, ruff, git

실행 환경

항목	내용
Python	requirements.txt 고정 버전으로 재현 가능
httpx	0.28.1
pydantic	2.13.4
pandas	2.3.3
pyarrow	22.0.0
pytest / pytest-asyncio	8.4.2 / 1.2.0
ruff	0.14.5

2. 프로그램 실행 결과

python -m app.main 실행 화면입니다. API 3개 동시 수집 → Pydantic v2 검증 → (고의로 이상값을 생성함) ValidationError 예외 처리 시연 → 날씨/국가/IP 컨텍스트 병합 → CSV·Parquet 저장 및 쓰기·읽기 성능 측정까지 전체 파이프라인이 순서대로 출력됩니다.

```
python -m app.main

=== 1. API 3개 동시 수집 (asyncio.gather) ===
수집 원료: weather, country, ip_location 3건 모두 응답 확인

=== 2. Pydantic v2 스키마 검증 ===
검증 원료: weather=72건, country=Korea (Republic of)(Seoul), ip=8.8.8.8->Ashburn

=== ValidationError 예외 처리 시연 (고의로 만든 이상값) ===
[예상된 오류] {'time': '2026-08-06T00:00', 'temperature_2m': 999, 'precipitation_probability': 10} ->
[{'type': 'less_than_equal', 'loc': ('temperature_2m',), 'msg': 'Input should be less than or equal to 50', 'input': 999,
[예상된 오류] {'time': '2026-08-06T01:00', 'temperature_2m': 20, 'precipitation_probability': 150} ->
[{'type': 'less_than_equal', 'loc': ('precipitation_probability',), 'msg': 'Input should be less than or equal to 100', 'input': 150}]

=== 3. 날씨 + 국가 + IP 컨텍스트 병합 ===
병합 원료: 72행

=== 4. CSV / Parquet 저장 및 쓰기·읽기 성능 측정 ===
CSV      : 쓰기 0.001359초 / 읽기 0.000896초 / 6541 bytes
Parquet  : 쓰기 0.005469초 / 읽기 0.00514초 / 7009 bytes

=== 5. 완료 ===
CSV: /tmp/work/판교_7반_임유리_day1종합실습/data/output/seoul_weather_report.csv
Parquet: /tmp/work/판교_7반_임유리_day1종합실습/data/output/seoul_weather_report.parquet
성능 결과: /tmp/work/판교_7반_임유리_day1종합실습/data/output/performance_result.json

[Day 1 종합실습] 파이프라인 종료
```

그림 1. python -m app.main 실행 결과

수집·검증 결과 요약

항목	결과
Open-Meteo (날씨)	72건 수집, 72건 검증 통과 (3일 × 24시간)
Countries.dev (국가정보)	Korea (Republic of) / 수도 Seoul / 검증 통과
ip-api (IP 위치)	8.8.8.8 → Ashburn, Virginia, US / 검증 통과
ValidationError 예외 처리 시연	기온 999도, 강수확률 150% 이상값 2건 → 정상적으로 예외 발생·포착 확인

3. CSV / Parquet 저장 및 성능 비교

검증을 통과한 72행(날씨 시점별 + 국가/IP 컨텍스트)을 CSV와 Parquet 두 형식으로 저장하고, 쓰기(write)·읽기(read) 시간을 time.perf_counter()로 측정했습니다.

형식	쓰기 시간(초)	읽기 시간(초)	파일 크기(byte)
CSV	0.001359	0.000896	6541
Parquet	0.005469	0.00514	7009

이번 실습 데이터는 72행 규모로 매우 작아서, 오히려 CSV가 Parquet보다 쓰기·읽기 모두 더 빨랐고 파일 크기도 더 작았습니다(Parquet는 컬럼 메타데이터·압축 스키마 등의 고정 오버헤드가 있어 소규모 데이터에서는 상대적으로 불리합니다). 일반적으로 알려진 "Parquet가 더 빠르고 작다"는 것은 수만~수백만 행 이상의 대용량·컬럼 지향 조회에서 성립하는 경향이라고 볼 수 있으며, 이번 실습 규모에서는 반대의 결과가 나온 점이 데이터 규모와 포맷 선택의 관계를 보여줬다고 할 수 있습니다.

4. 테스트 · 코드 품질 · Git 이력

pytest -v (11 건 전체 통과)

```
pytest -v

===== test session starts =====
platform linux -- Python 3.10.12, pytest-8.4.2, pluggy-1.6.0 -- /tmp/work/판교_7반_임유리_day1종합실습/.venv/bin/python3
cachedir: .pytest_cache
rootdir: /tmp/work/판교_7반_임유리_day1종합실습
configfile: pyproject.toml
testpaths: tests
plugins: anyio-4.14.2, asyncio-1.2.0
asyncio: mode=auto, debug=False, asyncio_default_fixture_loop_scope=None, asyncio_default_test_loop_scope=function
collecting ... collected 11 items

tests/test_api_client.py::test_fetch_all_calls_three_urls_concurrently PASSED [ 9%]
tests/test_models.py::test_weather_hour_accepts_valid_range PASSED [ 18%]
tests/test_models.py::test_weather_hour_rejects_out_of_range_probability PASSED [ 27%]
tests/test_models.py::test_country_info_rejects_non_positive_population PASSED [ 36%]
tests/test_models.py::test_ip_location_rejects_failed_status PASSED [ 45%]
tests/test_pipeline.py::test_extract_weather_rows_zips_three_arrays PASSED [ 54%]
tests/test_pipeline.py::test_extract_country_fields_picks_needed_keys PASSED [ 63%]
tests/test_pipeline.py::test_extract_ip_fields_converts_camel_case PASSED [ 72%]
tests/test_pipeline.py::test_validate_weather_rows_separates_valid_and_invalid PASSED [ 81%]
tests/test_pipeline.py::test_build_report_rows_broadcasts_context_columns PASSED [ 90%]
tests/test_storage.py::test_save_and_measure_creates_both_files_with_matching_row_counts PASSED [100%]

===== 11 passed in 0.23s =====
```

그림 2. pytest -v 실행 결과 (api_client / models / pipeline / storage 4개 파일, 11개 테스트 전체 PASSED)

ruff check . (오류 없음)

```
ruff check .

All checks passed!
```

그림 3. ruff check . 실행 결과 - "All checks passed!"

git log --oneline (커밋 이력)

```
git log --oneline

8a5cf35 주석 보강 반영 후 재실행한 파이프라인 결과 산출물 갱신
910949d main.py 주석 보강 및 회고 문구 최종본으로 교체
80bb017 practice2.py 스타일로 개념 설명 주석 보강 (config/models/api_client/pipeline/storage)
2933405 최종 git 커밋 이력 (git_log.txt) 갱신
de52cf2 실행결과 보고서 PDF 추가 (프로젝트 개요/실행결과/성능비교/테스트 · Git 이력/전체 코드/본인 의견)
88d91e6 제출용 git 커밋 이력 (git_log.txt) 저장
653378f 파이프라인 실행 결과 산출물 추가 (CSV/Parquet/성능 결과/원본 스냅샷)
45792d4 data/output 디렉터리 추적을 위한 .gitkeep 추가
1eb744d pytest 테스트 11건 추가 (api_client/models/pipeline/storage)
4847669 main 실행 진입점 작성 (제출용 헤더/회고 주석, 검증오류 시연 포함)
1739be0 CSV/Parquet 저장 및 쓰기 · 읽기 성능 측정 로직 작성
0300318 필드 추출 · Pydantic 검증 · 병합 파이프라인 로직 작성
c24d94d httpx+asyncio.gather() 기반 3개 API 동시 수집 클라이언트 작성
f917c46 Open-Meteo/Countries.dev/ip-api 대응 Pydantic v2 검증 모델 작성
2351b17 API 주소/파라미터/저장 경로 설정 모듈 (config.py) 작성
b63b9ff 프로젝트 초기 설정: requirements.txt, pyproject.toml, README.md 추가
```

그림 4. git log --oneline - 설정/모델/클라이언트/파이프라인/저장/메인/테스트/주석보강/산출물 단계별 커밋 이력

5. 본인 의견

구현하며 어려웠던 점

Countries.dev와 ip-api는 필드명이 서로 다른 스타일(예: ip-api의 `regionName` 같은 camelCase)이라, Pydantic 모델에 그대로 매핑하지 않고 `extract_country_fields` / `extract_ip_fields` 같은 별도 추출 함수를 만들어 필드명을 snake_case로 통일한 뒤 검증하는 구조로 정리했습니다. 또 ip-api는 HTTP 상태 코드가 200이어도 내부 status 필드가 "fail"일 수 있다는 점을 이번에 처음 확인했고, 이를 `field_validator`로 별도 검증해야 한다는 것을 배웠습니다.

비동기 수집의 장점

`asyncio.gather()`로 세 API를 동시에 호출해보니, 순서대로 `await` 하나씩 호출했을 때보다 전체 수집 시간이 눈에 띄게 줄었습니다. 동시에 기다린다는 개념을 코드로 직접 체감할 수 있어서 좋았던 것 같습니다.

Pydantic v2 검증

`Field(ge=..., le=...)`를 사용하니 if문으로 하나씩 검사하던 것보다 코드가 훨씬 짧고 의도가 분명해져서 좋았던 것 같습니다. 실제 API 응답은 대부분 정상 범위라 자연 발생 오류가 없었기 때문에, `demo_validation_error()`로 고의 이상값을 만들어 예외 처리 경로가 실제로 동작하는지 별도로 증명했습니다.

CSV와 Parquet 비교 결과

이번 72행 규모의 데이터에서는 CSV가 쓰기·읽기·파일 크기 모두에서 Parquet보다 유리했습니다. 처음에는 "Parquet가 항상 더 좋다"고 생각했는데, 실제로 측정해보니 데이터 규모가 작을 때는 Parquet의 스키마 압축 오버헤드가 오히려 손해라는 것을 직접 확인할 수 있었습니다. 데이터 규모에 따라 저장 포맷을 다르게 선택해야 한다는 깨달음을 얻을 수 있었습니다.

개선하고 싶은 점

지금은 세 API 중 하나라도 실패하면 `asyncio.gather()`가 바로 예외를 던지며 전체가 중단된다는 한계가 있습니다. 그렇기 때문에 `return_exceptions=True`를 활용해 "일부만 실패해도 나머지는 살리는" 구조로 바꾸면 더 좋아질 것 같다고 생각했습니다. 또한 지금은 위경도·조회 IP가 `config.py`에 고정값으로 들어있는데, CLI 인자나 환경변수로 받을 수 있게 하면 재사용성이 더 높아질 것이라고 생각했습니다.

코드 품질을 높이기 위한 제안

raw JSON → extract → validate → merge → save 로 단계를 분리해두었기 때문에, 각 단계를 독립적으로 pytest로 테스트할 수 있었습니다(특히 `api_client`는 `httpx.MockTransport`로 실제 네트워크 없이 테스트했습니다). 앞으로는 mypy 같은 정적 타입 검사기를 CI에 추가하면 Pydantic 모델과 함수 시그니처 사이의 타입 불일치를 실행 전에 더 빨리 잡아낼 수 있을 것 같다는 생각이 들었습니다.